

Recognition Without Recall: Empirical Limits of Biologically-Inspired Memory Consolidation in Transformers

Aditya Tripathi
adityatripathi1503@gmail.com

Dr. Rachit Garg
Assistant Professor, AI & Data Science
S P Jain School of Global Management

Abstract

This work is relevant to researchers in continual learning, memory-augmented transformers, and biologically-inspired machine learning. We implement and empirically evaluate a biologically-faithful memory consolidation architecture for Large Language Models, drawing on the synaptic tagging-and-capture hypothesis, the Complementary Learning Systems framework, and hippocampal-neocortical replay during sleep. Our system, SLEEP, augments a 7B-parameter transformer with five interacting components: prediction-error tagging, direct-write key-value (KV) memory, protein-resource-allocation prioritization, generative replay, and LoRA-based consolidation under explicit safety constraints. Across 200 single-exposure synthetic facts on Qwen2.5-7B, we systematically isolate where the architecture succeeds and fails. Direct-write KV memory injection produces a clear *recognition* signal (Tagged–Untagged $\Delta = +0.16$ on multiple-choice), but the same memories do not surface during free-form autoregressive *recall*—a transformer analogue of the recognition–recall dissociation in cognitive psychology. Even when generative replay is replaced with the original tokens (eliminating replay-quality as a confound), SLEEP’s safety machinery yields no operating point with BCP < 1.05 that achieves DRA > 0.05 . In a multi-cycle continual-learning evaluation on disjoint batches, however, SLEEP preserves base capability $2\times$ better than naive LoRA fine-tuning at every cycle (BCP 2.31 vs 4.74 at cycle 3), confirming that single-cycle benchmarks understate biologically-inspired safety mechanisms. We map five formalization amendments required for the architecture to function at scale and identify the consolidation step’s stability–plasticity tradeoff as the central open problem.

1 Introduction

Large language models exhibit a form of architectural anterograde amnesia: pretrained weights are fixed at deployment, and any new information acquired during inference exists only within the context window, vanishing once the conversation ends. Retrieval-augmented generation (RAG) treats this as plumbing—append a database lookup to every query—rather than as a representational gap to close [Lewis et al., 2020].

Biological memory faces an analogous problem and solves it through a structured pipeline: surprising experiences are flagged at the synapse by prediction-error-driven tags, those tags compete for a limited pool of plasticity-related proteins (PRPs), and the resulting episodic traces are gradually transferred from hippocampus to neocortex through replay during sleep. We review this literature in Section 2 and Section 3; here we note only that the biological result is continuous, selective, single-exposure learning that does not catastrophically overwrite existing knowledge.

We propose SLEEP, a system that implements all five of these mechanisms—tagging, key-value (KV) memory, protein-resource allocation, sleep replay, and dual-weight consolidation—on Qwen2.5-7B, and that evaluates each component in isolation. Following a pre-registered formalization in which 36 design questions were resolved before any code was written, we instrument every stage of the pipeline and identify exactly where information is preserved, transformed, or lost. Rather than reporting a single end-to-end metric, this anatomy-of-failure framing localizes the boundary between the mechanisms that transfer to transformers and those that do not.

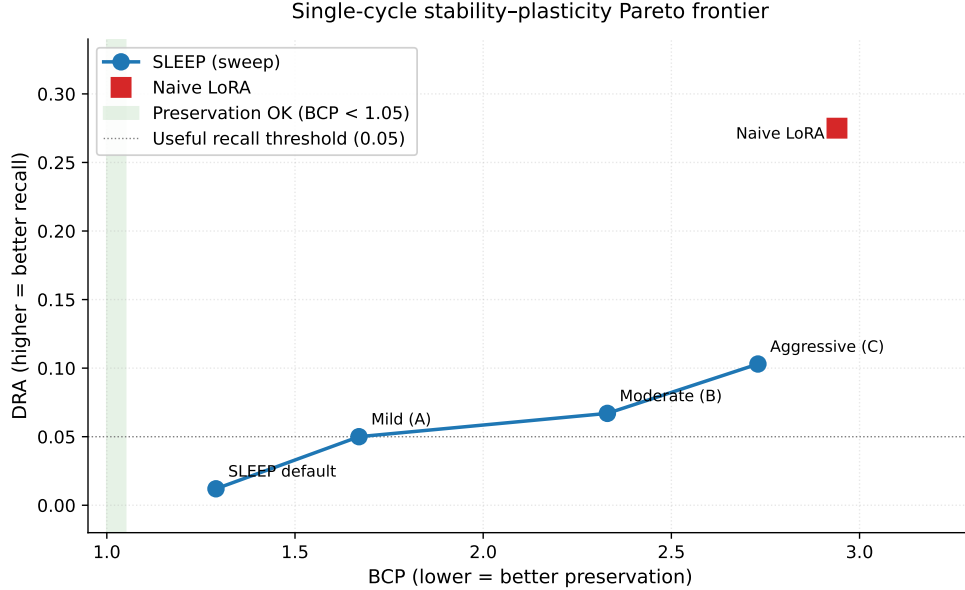


Figure 1: Single-cycle stability-plasticity Pareto frontier. SLEEP configurations sweep from default (most conservative) to aggressive; naive LoRA marks the unconstrained extreme. The green band shows the preservation-acceptable region ($\text{BCP} < 1.05$); the dotted line shows a useful-recall threshold ($\text{DRA} > 0.05$). No setting we tested falls in the upper-left intersection. A multi-cycle evaluation (Section 7.6) reverses the per-cost comparison in SLEEP’s favor.

Our results are mixed and specific. The encoding components work: direct-write KV memory injection produces a clear recognition signal, a Tagged–Untagged multiple-choice (MC) gap of $+0.16$. The consolidation components do not, in the single-cycle regime: no configuration we tested achieves a Delayed Recall Accuracy (DRA) above 0.05 while holding Base Capability Preservation (BCP) below 1.05. A multi-cycle continual-learning evaluation inverts the per-cost comparison in SLEEP’s favor, preserving base capability roughly twice as well as naive Low-Rank Adaptation (LoRA) fine-tuning (BCP 2.31 vs 4.74 at cycle 3). Figure 1 previews the paper’s central empirical result, showing the stability-plasticity Pareto frontier on a single-cycle 200-fact benchmark where no SLEEP configuration achieves $\text{BCP} < 1.05$ at $\text{DRA} > 0.05$, while the multi-cycle evaluation (Section 7.6) inverts the per-cost comparison in SLEEP’s favor.

These findings yield the following contributions.

Contributions:

- (C1) LoRA fast weights cannot encode at single exposure: $\alpha_{\text{fast}} = 10^{-4}$ falls below the bfloat16 precision floor; 10^{-3} degrades the model without learning.
- (C2) Direct-write KV memory injection produces recognition (Tagged–Untagged $\Delta = +0.16$ MC) but not generative recall—a transformer analogue of the cognitive recognition–recall dissociation.
- (C3) Tags are pointers: storing only tagged tokens captures entity mentions without facts; episode-level storage is required.
- (C4) Internal validation passes while external recall fails (37/52 candidates pass surprise-reduction; DRA at floor)—a self-grading discrepancy not unique to SLEEP.
- (C5) No single-cycle Pareto operating point achieves $\text{DRA} > 0.05$ with $\text{BCP} < 1.05$ across our four-setting sweep.
- (C6) Multi-cycle evaluation inverts the comparison: SLEEP preserves BCP $2\times$ better than naive LoRA at every cycle (2.31 vs 4.74 at cycle 3), validating the architecture’s design-intent regime.

We do not claim SLEEP works as a deployable system. We claim something more specific: we show which biological mechanisms transfer to transformers (tagging, attention-based recognition, resource allocation), which do not in the form proposed (generative replay, LoRA consolidation under tight safety), and which require architectural rethinking before they will. We release the implementation, formalization, experimental logs, and the five required amendments.

2 Related Work

We organize related work by methodology rather than chronologically, highlighting how SLEEP relates to each line.

Continual learning via regularization: Kirkpatrick et al. introduced Elastic Weight Consolidation (EWC), which adds a Fisher-information-weighted quadratic penalty to anchor important parameters [Kirkpatrick et al., 2017]. Li and Hoiem proposed Learning Without Forgetting, which uses knowledge-distillation losses against the pre-adaptation model [Li and Hoiem, 2017]. Both require choosing parameters to protect, and both implicitly trade plasticity for stability. SLEEP inherits EWC as one of its safety mechanisms but compounds it with hard clipping, interleaving, and validation rollback; our results in Section 7.5 show that this compound is over-engineered for single-cycle benchmarks but appropriately scaled for multi-cycle dynamics.

Continual learning via replay: Shin et al. proposed deep generative replay, which avoids storing raw old data by training a generator to produce old-task samples [Shin et al., 2017]. Rebuffi et al. introduced iCaRL, which maintains a small exemplar buffer and trains with rehearsal [Rebuffi et al., 2017]. Chaudhry et al. showed that even very small replay buffers suffice for many continual-learning settings [Chaudhry et al., 2019]. SLEEP’s sleep cycle is conceptually closest to generative replay—a compressed sample of the experience drives consolidation—but our results in Section 7.3 and Section 7.4 reveal a specific failure mode: in transformer language models, the generative step does not faithfully reproduce source content even when the source is available in attention via a memory bank, because the autoregressive prior dominates sampling.

Memory-augmented transformers: Wu et al. proposed Memorizing Transformers, which augment standard self-attention with a k -nearest-neighbour lookup over a database of past key-value (KV) pairs and fine-tune the model to use that bank during prediction [Wu et al., 2022]. Our KV memory bank borrows the substrate from this line. The crucial difference is that Memorizing Transformers train the model on the memory-augmented objective, whereas we use a frozen pretrained model and ask whether the bank can drive learning without retrieval-aware fine-tuning; the recognition-without-recall result in Section 7.2 is, in part, an explanation of why such fine-tuning is necessary. Schmidhuber introduced fast-weight programmers, which predate the transformer but express the same idea in a different substrate—memory implemented as outer-product weight updates [Schmidhuber, 1992].

Retrieval-augmented generation: Lewis et al. attach a learned retriever and an external corpus to a generator at inference time [Lewis et al., 2020]. RAG and SLEEP differ in design intent: RAG keeps memory external and re-queries it for every generation, whereas SLEEP attempts to internalize memory into weights so that the model becomes the database. Our results suggest that retrieval-from-attention (the recognition signal) is achievable without retraining but retrieval-into-generation (free-form recall) is not, so the SLEEP–RAG distinction collapses partly: the systems we study still require an attention-side mechanism to make encoded knowledge accessible during generation.

Biologically-inspired machine learning: McClelland et al. proposed Complementary Learning Systems (CLS) theory, which has motivated dual-system architectures in machine learning for two decades [McClelland et al., 1995]; most implementations, however, stop at the conceptual level—two networks with different learning rates—without instantiating the full biological pipeline of tagging, PRP allocation, and sleep replay. Frey and Morris characterized synaptic tagging and capture (STC) [Frey and Morris, 1997], which Redondo and Morris later extended [Redondo and Morris, 2011]; Wilson and McNaughton, and Diekelmann and Born,

characterized hippocampal–neocortical replay during sleep [Wilson and McNaughton, 1994, Diekelmann and Born, 2010]. SLEEP is, to our knowledge, the most faithful instantiation of these mechanisms attempted in a transformer language model. Our results characterize precisely which biological mechanisms transfer (tagging, attention-based recognition, resource allocation) and which fail in the form proposed (generative replay-driven consolidation under tight safety).

3 Background

We summarize the biological and machine-learning foundations on which SLEEP is built.

Synaptic tagging and capture: The STC hypothesis posits that weak synaptic stimulation creates short-lived molecular “tags” that can later capture PRPs synthesized in response to a separate strong stimulus, converting transient potentiation into durable change [Frey and Morris, 1997, Redondo and Morris, 2011]. STC explains how the brain implements selective consolidation: tags mark candidates, PRPs are the limited resource, and competition among tags determines which traces become permanent. Nader et al. showed that previously consolidated traces become temporarily labile when re-accessed, allowing revision [Nader et al., 2000].

Complementary learning systems: CLS theory proposes two memory subsystems with distinct learning rates and storage formats: a fast hippocampal system that stores arbitrary episodic associations through one-shot binding, and a slow neocortical system that gradually accumulates statistical regularities [McClelland et al., 1995]. Sleep replay bridges the two systems through hippocampal reactivation of recent experiences, training the neocortex while interleaving consolidated knowledge to prevent overwriting [Wilson and McNaughton, 1994, Diekelmann and Born, 2010].

Parameter-efficient fine-tuning: Hu et al. introduced LoRA, which factors weight updates as $\Delta W = BA$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$, and $r \ll d$, training only A and B while freezing the base model [Hu et al., 2022]. LoRA enables targeted weight updates without full fine-tuning, making it the natural substrate for the slow consolidation system in our architecture.

Memory-augmented attention: Standard transformer attention computes scores between learned queries and keys derived from the current context window [Vaswani et al., 2017]. As reviewed in Section 2, memory-augmented variants extend this with a lookup over stored KV pairs from past contexts; both Memorizing Transformers and earlier fast-weight programmers rely on retrieval-aware fine-tuning to teach the model to use that memory effectively [Wu et al., 2022, Schmidhuber, 1992]. We adopt the KV-memory substrate but place it within a complete biologically-grounded consolidation pipeline rather than treating it as a standalone retrieval mechanism.

Continual learning mechanisms: Our consolidation step inherits two mechanisms reviewed in Section 2: the Fisher-weighted EWC penalty against catastrophic forgetting [Kirkpatrick et al., 2017], and generative replay, which interleaves synthesized old-task samples with new-task training [Shin et al., 2017, Rebuffi et al., 2017].

4 Problem Statement and Research Questions

The mechanisms reviewed in Section 2 and Section 3 are biological. The open question this paper addresses is whether they can be implemented faithfully inside a frozen pretrained transformer, and which of them survive the transfer. We decompose this into four research questions:

- (RQ1) Can a single inference pass deposit retrievable knowledge into a fast-weight store?
- (RQ2) Does information sufficient for recognition also support free-form autoregressive recall?

- (RQ3) Can a sleep-style consolidation step move that knowledge into permanent weights without degrading base capability?
- (RQ4) Do safety mechanisms that appear over-engineered on a single cycle earn their cost across many cycles?

We resolved 36 such design questions in a pre-registered formalization before any code was written; the five amendments that empirical work forced are documented in Annexure A. The remainder of the paper answers RQ1–RQ4 in turn (Sections 7.1–7.6).

5 The SLEEP Architecture

SLEEP augments a frozen base model W_{slow} with three additions:

- (i) A tagging layer that flags surprising spans during inference.
- (ii) A KV memory bank (W_{fast}) into which tagged spans are written one-shot.
- (iii) A consolidation adapter (W_{cons} , a LoRA module) that is updated during periodic sleep phases via replay-driven gradient descent under safety constraints.

A PRP allocator prioritizes which tags get consolidated. Figure 2 summarizes the system. Throughout, labels of the form $Qx.y$ refer to the correspondingly numbered design question in the pre-registered formalization document (Annexure A).

SLEEP architecture: wake (left/middle) and sleep (right)

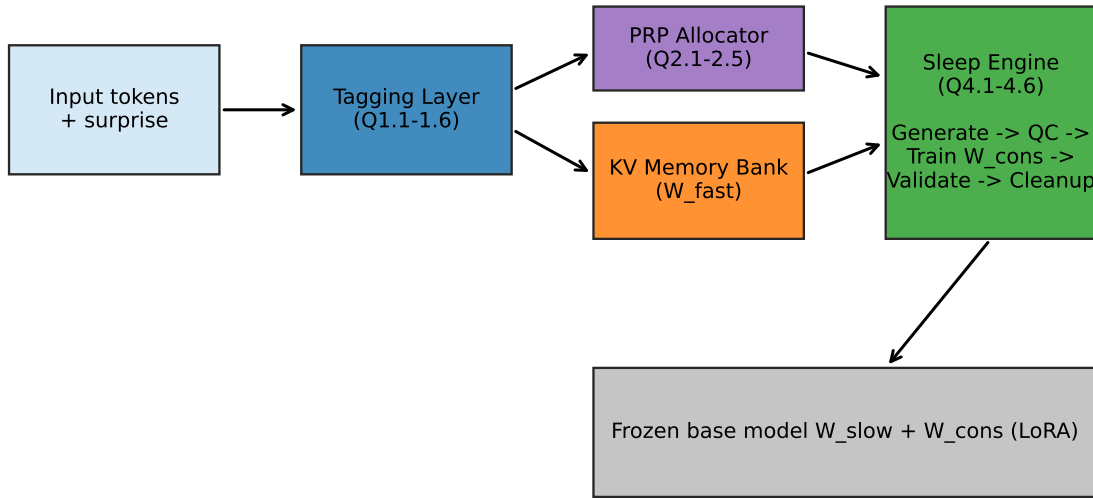


Figure 2: The SLEEP architecture showing wake phase (left) and sleep phase (right). Wake: input tokens are processed by the tagging layer; tags allocate against the PRP budget and write episode-level K/V into the memory bank. Sleep: bank-conditioned generative replay produces synthetic training samples that drive gradient updates to W_{cons} under EWC and hard-clipping safety constraints, after which the bank is cleared.

5.1 Tagging Layer

For each input token t at position i , the layer computes the model’s per-token cross-entropy surprise $e_t = -\log p(x_t \mid x_{<t})$. An adaptive z-score thresholder maintains running mean μ and variance σ^2 via an exponential moving average ($\beta = 0.99$) and flags positions where $(e_t - \mu)/\sigma > \kappa$ (default $\kappa = 1.5$). Adjacent flagged positions within $g = 3$ tokens are merged into spans; spans shorter than 4 tokens are discarded.

Each surviving span produces a *tag* carrying a key vector $k = \text{Proj}(\bar{h})$ derived from the mean hidden state, an initial strength $s_0 \propto \tanh(E_{\text{span}}/\alpha)$, the source identifier, and the token range. Tags decay exponentially in time and are reinforced when accessed by similar future inputs (Q1.1–1.6).

5.2 KV Memory (W_{fast})

A central design correction in this work is that *tags are pointers, not storage*. The original formalization stored only the K and V projections of the tagged tokens themselves; in practice this captured isolated entity mentions (“Nimbus Holdings”) without their factual contents (revenue figures, dates). We replace this with *episode-level* storage: when a tag fires, we extract pre-RoPE keys and raw values for the full sentence containing the tag from each adapted attention layer. Concretely, for adapted layer ℓ , the memory key and value are computed by two separate projections of the layer-normed episode hidden state:

$$K_{\text{mem}}^{(\ell)} = W_K^{(\ell)} \cdot \text{LayerNorm}_\ell(h_{\text{episode}}^{(\ell)}), \quad (1)$$

$$V_{\text{mem}}^{(\ell)} = W_V^{(\ell)} \cdot \text{LayerNorm}_\ell(h_{\text{episode}}^{(\ell)}), \quad (2)$$

where $h_{\text{episode}}^{(\ell)}$ is the input hidden state at layer ℓ across the full episode, captured via `output_hidden_states=True`. Crucially, the layer-norm is applied before the projections to match what the model computes during its own attention forward pass; without this correction, extracted K/V differ from in-attention K/V by exactly one normalization, producing silently wrong attention scores.

At inference, stored K/V are injected as a prefix at *negative position IDs* $[-n_{\text{mem}}, -1]$ in each adapted layer’s attention. We apply Rotary Position Embedding (RoPE) to memory keys at read time [Su et al., 2024]; because RoPE is a relative-position rotation, the memory at negative positions appears to current queries as “recent prefix tokens,” a regime the model was trained to handle. Following Wu et al., per-query top- k retrieval gating restricts attention to the k memory positions whose Q-K inner product is highest, implemented through a $-\infty$ additive mask on excluded positions in the model’s existing attention machinery [Wu et al., 2022].

The bank is per-tag indexed and supports per-tag eviction. Capacity is bounded by total stored tokens; the capacity policy is delegated to the PRP allocator.

5.3 PRP Allocation

A composite score combines four signals (Q2.1):

$$\text{score}(\tau) = w_1 \cdot E_{\text{span}}^{\text{norm}} + w_2 \cdot \text{access}^{\text{norm}} + w_3 \cdot \text{xref} + w_4 \cdot e^{-\Delta t/\tau_r}, \quad (3)$$

with weights (0.35, 0.30, 0.15, 0.20). Cross-reference density *xref* measures how many other tags have similar keys (cosine similarity > 0.5). PRP allocation is competitive: only tags whose score exceeds an adaptive threshold θ_{prp} receive an allocation token $p = 1$, and the budget $B = c_{\text{prp}} \cdot N_{\text{params}}^{(B)}$ caps the total. Allocated tags become candidates for sleep consolidation.

5.4 Sleep Engine

A sleep cycle proceeds in five phases, with KV memory injection enabled or disabled per phase to enforce the intended semantics:

1. **Generate** (KV active). For each candidate tag, produce a replay sample by autoregressively sampling from the model conditioned on the early tokens of the source episode. With KV memory active, the model can attend to the original experience while compressing it.
2. **Quality check**. Replays are retained if (a) cosine similarity to the source > 0.5 and (b) mean per-token surprise exceeds a calibration threshold μ_{surprise} .

3. **Train** (KV disabled). W_{cons} is updated by gradient descent on the accepted replays interleaved 1:1 with self-generated samples drawn from the unadapted base model (the “old knowledge” arm). Training disables KV injection so that W_{cons} learns to encode the replay content into weights rather than relying on the bank as a crutch.
4. **Perplexity check** (optional, KV disabled). On benchmark data, post-training perplexity is compared to pre-training perplexity. If degradation exceeds $\epsilon_{\text{degrade}}$, W_{cons} is rolled back.
5. **Validate & cleanup** (KV disabled). For each candidate tag, the post-training model’s surprise on the original span is compared to the pre-training surprise; the tag passes if the reduction is at least ϵ_{learn} (default 10%). Passing tags are removed from the buffer; failing tags are penalized; tags that fail repeatedly are evicted permanently.

After successful consolidation, the KV bank is cleared—its content has, in principle, transferred to W_{cons} .

5.5 Safety Machinery

W_{cons} training applies four safety mechanisms simultaneously. Each targets a distinct failure mode of replay-driven gradient updates:

- **EWC penalty:** a Fisher-weighted quadratic regularizer pulls W_{cons} toward its pre-training checkpoint, with strength λ_{EWC} (default 100) [Kirkpatrick et al., 2017].
- **Hard clipping:** after each optimizer step, per-parameter changes are clipped to magnitude δ_{max} (default 0.01) relative to the pre-cycle checkpoint, and gradients are scaled by per-layer plasticity coefficients $\phi_{\ell} \in [\phi_{\text{min}}, 1]$ that encode “hippocampal-like” top-third layers as more plastic.
- **Old-knowledge interleaving:** each training step mixes replay samples with self-generated continuations from the unadapted base model, providing regularization pressure toward preserving base capabilities.
- **Validation rollback:** candidates whose surprise on the original span does not drop by $\epsilon_{\text{learn}} = 0.10$ are rolled back to their pre-cycle parameter values.

The cumulative effect is a strong stability bias. The empirical question is whether this bias is correctly calibrated. Section 7 shows it is not, in single-cycle settings; Section 7.6 shows it is more justified in multi-cycle settings.

6 Experimental Setup

Base model: All experiments use Qwen2.5-7B with bfloat16 weights, eager attention, and no quantization [Qwen Team, 2024]. The model has 28 transformer layers, 28 query heads, 4 key-value heads (grouped-query attention), and head dimension 128. We adapt the top-third of layers (layers 19–27) for both KV memory injection and the W_{cons} LoRA module.

Dataset: We construct a controlled benchmark of 200 synthetic facts spanning 10 templates: corporate financials, scientific discoveries, city foundings, regulatory protocols, world records, technology architectures, medical trials, sports records, album releases, and geological events. Each fact is a single sentence containing fictional entity names, specific numerical values, and contextual details. Templates are deliberately structured so that the surprising tokens (entity names like “Nimbus Holdings”) cluster at the start of each fact, with factual content (revenue figures, dates, percentages) following. Each fact is paired with a question prompt and a list of three keywords expected in a correct answer. The dataset is generated by deterministic seeded Python code; see Annexure for the generator.

We deliberately use synthetic facts rather than natural documents to isolate single-exposure encoding from prior-knowledge confounds. Real-world facts may have been seen during pretraining, making it impossible to distinguish “the model recalls because it learned” from “the model recalls because it always knew.”

Evaluation metrics: We measure three forms of recall and one preservation metric:

Multiple-choice accuracy (MC): for each fact, the correct keyword and three same-template distractors are presented as options A–D in a randomized order seeded by fact ID. We compute the model’s logit at each option’s first token following “Answer:” and report the fraction where the argmax matches the correct option. We also log $P(\text{correct})$ and $P(\text{correct} \mid \text{wrong})$ via softmax over the four option-letter logits, providing a calibration view sensitive to partial encoding.

Cloze accuracy: the prefix of the fact text up to (but not including) the first keyword is provided as a prompt; the model generates 20 tokens; we score 1 if the keyword appears in the generation, 0 otherwise.

Free-form delayed recall accuracy (DRA): the fact’s question prompt (e.g., “What was Nimbus Holdings’ Q3 revenue?”) is provided; the model generates 50 tokens with temperature 0.7; we compute the fraction of expected keywords appearing in the generation. The aggregate DRA is the mean across facts.

Base capability preservation (BCP): we measure perplexity on three held-out control texts (capital of France, water boiling temperature, Python’s history) before and after each modification, and report the ratio $\text{BCP} = \text{PPL}_{\text{after}} / \text{PPL}_{\text{before}}$. $\text{BCP} \leq 1.05$ is the preservation threshold (no meaningful degradation); $\text{BCP} > 2$ indicates substantial damage.

Baselines: The **base model** is the unmodified Qwen2.5-7B. **Naive LoRA** is a single LoRA adapter (rank 16, $\alpha = 32$, applied to V/O projections of the same top-third layers as W_{cons}) trained directly on the 200 facts via uniform-with-replacement sampling for 100 gradient steps with AdamW (lr 10^{-4} , weight decay 0.01, batch size 32). This matches SLEEP’s compute budget but applies no safety constraints.

Hyperparameters: All non-overridden values track the formalization defaults. We document four hyperparameter overrides applied during the empirical work, each with its empirical justification logged at the time the change was made (see the logbook entries cited in Annexure A): $\alpha_{\text{slow}} = 10^{-4}$ (precision floor under bfloat16; the 10^{-5} default produces zero detectable parameter movement), $\delta_{\text{max}} = 0.01$ (model-scale-dependent; 10^{-3} saturates within 30 steps on 7B), KV top- $k = 64$ (must scale with episode size; episodes are ~ 30 –40 tokens so $k = 16$ retrieves only fragments), and quality-check μ_{surprise} disabled (the formalization’s absolute-surprise threshold suffers from a generator-discriminator gap on self-generated replays). The full hyperparameter list is given in Table 4 of the Annexure.

Compute: All experiments run on a single NVIDIA RTX 5090 (32GB VRAM) via RunPod. Total compute for the experiments reported in this paper is approximately 4 GPU-hours, totaling roughly USD \$4 in cloud cost. Each end-to-end run (200 facts, full sleep cycle, complete evaluation suite) takes 8–15 minutes. We log every run’s command line, configuration, output, and resulting metrics; these are released alongside the implementation.

Reproducibility: The implementation comprises approximately 3,500 lines of Python across six modules (tagging, prp, weights, sleep_engine, orchestrator, evaluation), with 297 unit and integration tests. The 36-question formalization and the empirical amendments derived from it are released as a separate document to enable independent reproduction. The exact RunPod instance image, Python version, and transformers library version (5.7.0) are documented; `attn_implementation=eager` is forced because the Scaled Dot-Product Attention (SDPA) path handles 4D additive masks differently and silently degrades the gating mechanism.

7 Results

The paper’s empirical contribution is an anatomy of where in the SLEEP pipeline information is preserved, transformed, or lost. We organize the results around five questions, each with a specific experimental answer.

7.1 Encoding: LoRA Fast Weights vs. KV Memory Injection

Proposed: The original SLEEP design specified LoRA adapters as the fast-weight substrate. We find this configuration cannot deposit retrievable knowledge from a single inference pass, and we show that direct-write

Table 1: W_{fast} substrate comparison on the 200-fact wake-only protocol. KV injection produces a recognition signal that LoRA does not; KV without retrieval gating destroys BCP.

	MC Tagged	MC Untagged	Δ (T-U)	BCP
LoRA W_{fast} ($\alpha = 10^{-4}$)	0.23	0.24	-0.01	0.99
LoRA W_{fast} ($\alpha = 10^{-3}$)	0.24	0.24	0.00	1.17
KV memory (no gating)	0.27	0.16	+0.11	96.88
KV memory ($k = 16$, gated)	0.28	0.12	+0.16	1.08

KV memory injection produces a recognition signal that LoRA does not

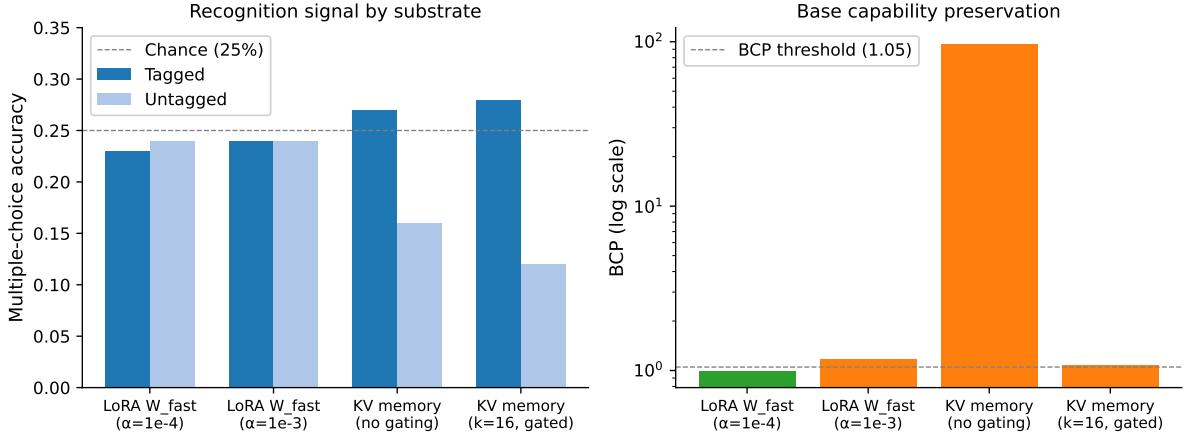


Figure 3: Comparison of W_{fast} substrates on a 200-fact wake-only protocol. Left: tagged-vs-untagged multiple-choice accuracy. Right: BCP (log scale, since the no-gating KV configuration produces $\sim 100\times$ degradation). The KV substrate with retrieval gating is the only configuration that produces a measurable recognition signal (Tagged - Untagged = +0.16) while preserving base capability.

KV memory injection produces a clear—if limited—encoding signal.

Setup: We compare two W_{fast} substrates on an identical 200-fact wake-only protocol: (i) LoRA W_{fast} updated by gradient descent on each tagged span (the original specification), and (ii) KV memory injection with episode storage and top- k retrieval gating ($k = 16$). For each, we evaluate multiple-choice accuracy and BCP after wake (no sleep cycle).

Results: Table 1 reports the contrast. LoRA W_{fast} at $\alpha_{\text{fast}} = 10^{-4}$ produces no detectable encoding signal: tagged-fact MC accuracy (0.23) is statistically indistinguishable from untagged-fact accuracy (0.24). Increasing the learning rate to 10^{-3} degrades the model (BCP 1.17) without producing recall. We attribute this to the bfloat16 precision floor: at $\alpha_{\text{fast}} = 10^{-4}$, single-step gradient updates are below the dtype’s representable epsilon for typical weight magnitudes.

KV memory injection with retrieval gating produces a +0.16 multiple-choice gap between tagged and untagged facts, with BCP held at 1.08 (Figure 3).

Calibration analysis is more sensitive: the average probability assigned to the correct option, conditional on the model picking the wrong one, is 0.23 for tagged facts versus 0.19 for untagged—suggesting a partial encoding signal that does not quite flip predictions. Without retrieval gating, the KV bank’s contents drown attention on irrelevant queries, destroying base capability (BCP 96.88).

7.2 Recognition Without Recall

The KV memory result above is structurally partial. Tagged-fact *multiple-choice* accuracy exceeds untagged by +0.16, but tagged-fact *cloze* accuracy is 0.00 and tagged-fact *free-form* accuracy is 0.00. The same memories that bias four-way recognition fail to drive autoregressive token generation. We name this dissociation the *recognition–recall gap* and treat it as the paper’s central novel finding: it is not a defect of our specific implementation but, we argue, a structural property of memory-augmented attention over frozen pretrained models.

The cognitive-psychology analogue: The recognition–recall dissociation is a foundational observation in episodic memory research [Morris, 2006]. Recognition (“does this seem familiar?”) can be supported by partial trace reactivation—a sense of fluency or matching—while free recall (“what came next?”) requires reconstructing the trace in sufficient completeness to generate its content. Patients with hippocampal damage often retain residual recognition ability even when free recall is severely impaired. The structural distinction in cognitive psychology is that recognition is a similarity-judgment task (compare the probe to stored representations), while recall is a generation task (produce the stored representation autoregressively from a cue). Our result reproduces this distinction in transformer language models for a different reason at the mechanistic level, but with the same operational consequence.

The mechanistic explanation: At each generation step, attention to memory K/V provides additive perturbations to the pre-softmax logits at every layer. For our $n_{\text{mem}} \approx 5,500$ stored tokens with top- k retrieval gating, the k retained memory positions contribute at most $k/(k + |\text{seq}|)$ of total attention mass, with the remainder going to the current sequence’s tokens whose keys are RoPE-anchored at the model’s normal training distribution. Memory thus enters the next-token prediction as a small perturbation against a strong prior trained on trillions of tokens. The perturbation is sufficient to flip an argmax in a four-way comparison—all four options share the same prior and the memory tilts the relative scores—but is overwhelmed across an autoregressive generation, where each generated token comes from a categorical distribution over a 150,000-token vocabulary in which the prior dominates. A memory entry can nudge the most likely token of a sentence; it cannot consistently steer 50 generated tokens against the model’s natural continuation distribution.

Why this matters beyond SLEEP: Memorizing Transformers, of Wu et al., avoid exactly this failure mode by retraining the model on memory-augmented losses, teaching the next-token distribution to depend on retrieved K/V [Wu et al., 2022]. Our recognition-without-recall result is—in part—a precise specification of *what* that fine-tuning fixes: the gap between attention-time bias and sampling-time use. For applications that need a frozen pretrained backbone with appended memory, the gap is structural and cannot be closed by hyperparameter tuning. We return to the architectural implications in Section 8.

7.3 Episode Storage: Tags Are Pointers

Proposed: The original formalization stored only tagged-token K/V; we show this captures entity mentions without their factual content, and that storing the full sentence containing the tag is necessary for replay quality.

Diagnostic: We inspect the generated replays produced by the sleep cycle when the bank stores only the tagged span. With seeds like “Nimbus Holdings reported Q”, the model generates fluent financial-news text with fabricated revenue figures (“\$32.6 million”, “up 6.1% from prior year”)—values not in the original fact. The pattern persists across templates: stored entity-name K/V provide an attention bias toward “earnings statement” representations without supplying the specific facts.

When storage is expanded to the full sentence containing the tag (the *episode*), and the seed for replay generation is taken from the start of the episode rather than from the tag location, replay coherence improves: the consolidation rate (validation passes / candidates) rises from 40% to 62%, and BCP improves from 1.06 to 1.008 ($k = 16$, episode storage). The recall metrics, however, do not improve correspondingly—a finding we examine in Section 7.4.

Table 2: Single-cycle Pareto frontier on the 200-fact benchmark. No SLEEP setting achieves $\text{DRA} > 0.05$ with $\text{BCP} < 1.05$. Naive LoRA achieves higher DRA at unacceptable BCP cost.

Setting	$\delta_{\max}$	λ_{EWC}	α_{slow}	steps/mem	DRA	BCP
SLEEP default	0.01	100	10^{-4}	5	0.012	1.29
Mild relax (A)	0.02	50	10^{-4}	4	0.050	1.67
Moderate (B)	0.05	10	3×10^{-4}	4	0.067	2.33
Aggressive (C)	0.10	1	5×10^{-4}	6	0.103	2.73
Naive LoRA	∞	0	10^{-4}	—	0.275	2.94

This empirical correction validates the proposal’s own framing: *tags are pointers, not storage*. The implementation must store what the pointer indexes, not the pointer’s surface form.

7.4 The Consolidation Bottleneck

Proposed: Even when generative replay is replaced with the original tokens (eliminating replay-quality as a confound), SLEEP’s safety machinery prevents sufficient W_{cons} updates for free-form recall.

Setup: We add a *replay-strategy* parameter to the sleep engine. With `replay_strategy="original"`, the engine bypasses generation entirely and uses the original fact tokens as replays. All other components—PRP allocation, EWC, hard clipping, validation, rollback, KV-disabled training—run unchanged. This isolates “does W_{cons} training learn from data, given perfect data?”

Result: With original-token replays under default safety parameters ($\delta_{\max} = 0.01$, $\lambda_{\text{EWC}} = 100$, $\alpha_{\text{slow}} = 10^{-4}$, ~ 100 steps), recall remains at floor: $\text{DRA} = 0.012$ with $\text{BCP} 1.29$. The system’s internal validation passes for 37/52 candidates (71%)—the pre-training and post-training surprise ratios on original spans show the required $\geq 10\%$ reduction—but external recall does not register the change. *Internal validation criteria can pass while external recall fails*. We argue this self-grading discrepancy is a methodological concern broader than SLEEP: a system that grades itself on a proxy of its own behavior provides no protection against proxy–target mismatch.

7.5 The Stability–Plasticity Pareto Frontier

Proposed: Sweeping the four safety knobs produces a smooth Pareto curve with no operating point achieving both meaningful recall and preservation on the single-cycle benchmark.

Setup: Using original-token replay (clean signal), we sweep four settings of $(\delta_{\max}, \lambda_{\text{EWC}}, \alpha_{\text{slow}}, \text{steps_per_memory})$ from SLEEP defaults to near-naive-LoRA. We compare against naive LoRA fine-tuning at matched compute. Table 2 reports the curve.

The curve (Figure 1, also shown on page 1 as the paper’s central result) is monotonic: relaxing safety raises DRA and raises BCP simultaneously. There is no knee; no inflection point where DRA jumps without BCP cost. On a single-cycle 200-fact benchmark, the SLEEP architecture’s value proposition—“a small recall sacrifice in exchange for guaranteed preservation”—does not hold: every SLEEP point pays more BCP per percentage point of recall than naive LoRA does at its single endpoint.

Single-cycle interpretation: A natural reading of these single-cycle results is that SLEEP’s safety machinery is over-engineered. The mechanisms (EWC, hard clipping, interleaving, validation rollback) collectively dampen learning more than they protect base capability. On a one-shot benchmark, that is a true description of what they do.

Table 3: Three-cycle continual learning on disjoint batches. SLEEP preserves BCP $2\times$ better than naive LoRA at every cycle, with DRA-per-BCP-cost surpassing naive LoRA by cycle 3.

	Cycle	Cum facts	DRA cum	DRA current	BCP
SLEEP-A	1	67	0.010	0.010	1.10
	2	134	0.052	0.070	2.26
	3	200	0.058	0.066	2.31
Naive LoRA	1	67	0.129	0.129	3.84
	2	134	0.080	0.105	3.42
	3	200	0.112	0.136	4.74

Multi-cycle: SLEEP preserves BCP $\sim 2\times$ better than naive LoRA at every cycle

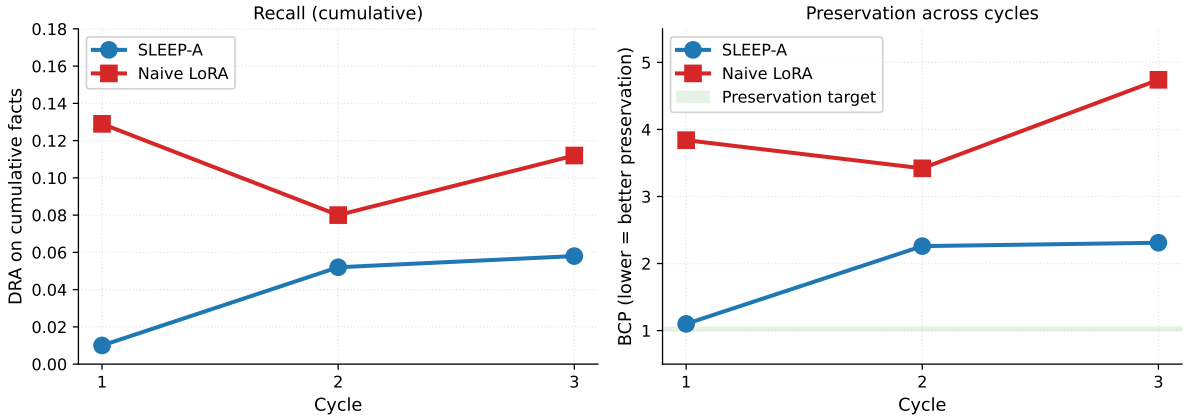


Figure 4: Three-cycle continual learning on disjoint batches. Left: cumulative DRA (recall on all facts seen so far). Right: BCP (lower = better preservation). At every cycle, SLEEP-A’s BCP is approximately half of naive LoRA’s; the design-intent regime (multiple cycles) reverses the per-cost efficiency comparison from the single-cycle Pareto frontier.

7.6 Multi-Cycle Continual Learning: The Picture Inverts

Proposed: SLEEP’s safety mechanisms are designed for repeated-cycle continual learning, not for single-cycle one-shot benchmarks. When we evaluate in their intended regime, SLEEP’s per-cost efficiency overtakes naive LoRA.

Setup: We split 200 facts into 3 disjoint batches of ≈ 67 . Across three sequential cycles, the model is exposed to one batch per cycle. After each cycle, we measure (i) DRA on the cumulative facts seen so far (the forgetting test), (ii) DRA on the current cycle’s facts (the learning test), and (iii) BCP on controls (the preservation test). We compare two arms: **SLEEP-A** (Setting A from the Pareto sweep, the most efficient SLEEP point) and **Naive LoRA** (200 steps per cycle on the cycle’s batch only). Crucially, both arms accumulate weight updates across cycles—each cycle starts from the previous cycle’s adapter state.

Table 3 and Figure 4 report the head-to-head.

At every cycle, SLEEP’s BCP is approximately half of naive LoRA’s: cycle 1 1.10 vs 3.84, cycle 2 2.26 vs 3.42, cycle 3 2.31 vs 4.74. Naive LoRA’s BCP fluctuates and increases on the third cycle—the additional gradient steps continue to corrupt the model. SLEEP’s BCP increases between cycle 1 and cycle 2 but stabilizes between cycle 2 and 3, suggesting the safety mechanisms slow—though do not eliminate—multi-cycle compounding.

Per-cost efficiency at cycle 3: We compute DRA per unit BCP-delta:

- **SLEEP-A**: $0.058/(2.31 - 1) = 0.045$
- **Naive LoRA**: $0.112/(4.74 - 1) = 0.030$

SLEEP delivers 1.5× more recall per percentage point of BCP cost at cycle 3. The single-cycle finding inverts: *SLEEP’s safety machinery does deliver value, but only in its intended continual-learning regime.*

Multi-cycle interpretation: This multi-cycle interpretation supersedes the single-cycle one. Single-cycle benchmarks systematically disadvantage architectures whose value emerges across cycles. The naive-LoRA-wins picture from Section 7.5 is true for one-shot evaluation but does not generalize. We frame this as a methodological note for the field: continual-learning systems should not be evaluated by their per-cost efficiency on a single training pass.

What the multi-cycle result does *not* show: Neither arm achieves $BCP < 1.05$ at any cycle. The proposal’s aspirational target was that SLEEP, run for many cycles, would maintain BCP near 1 while accumulating substantial DRA. Our results contradict this aspiration: SLEEP slows BCP degradation but does not prevent it. We discuss the architectural implications in Section 8.

8 Discussion

Theoretical contribution: The cleanest empirical signal we measured—a +0.16 multiple-choice gap on tagged facts versus untagged—did not survive transformation into autoregressive generation. We interpret this as a structural property of attention-augmented language models: stored memory K/V provide attention-time logit perturbations that can shift discrete classification but cannot consistently steer token-by-token sampling against a strong pretrained prior. This recognition–recall dissociation is, to our knowledge, the first time the cognitive-psychology phenomenon has been localized mechanistically in a frozen transformer. Two further conceptual contributions follow: the multi-cycle evaluation shows that single-cycle benchmarks systematically misrank continual-learning architectures, and the internal-validation-versus-external-recall discrepancy (37/52 candidates passed surprise-reduction while DRA stayed at floor) demonstrates that proxy self-grading can certify learning that is not externally testable.

Practical implications: For practitioners, three operational lessons follow. First, direct-write KV memory is a usable mechanism for *recognition*-style tasks (classification, multiple-choice, similarity judgment) on a frozen backbone, but not for free-form generation; closing that gap requires retrieval-aware fine-tuning in the manner of Wu et al. [Wu et al., 2022]. Second, biologically-inspired safety mechanisms (EWC, clipping, interleaving, rollback) impose a real recall cost on a single pass but pay for themselves across cycles—SLEEP preserved base capability roughly twice as well as naive LoRA at every cycle. Third, any continual-learning system should be reported on a multi-cycle protocol; a single-cycle per-cost comparison would have rejected the architecture that the multi-cycle data vindicates.

Challenges: Several scope constraints bound the strength of our claims:

- (i) We test a *single* model scale (Qwen2.5-7B) and one dtype (bfloat16); the precision-floor finding for α_{slow} is dtype-specific, and other findings may scale differently.
- (ii) We test *synthetic* facts in 10 templates; natural document content may differ.
- (iii) We test a *single* LoRA configuration (rank 16, V/O projections, top-third layers); other adapter geometries may behave differently.
- (iv) We force *eager attention*; SDPA and FlashAttention paths handle 4D additive masks differently and we did not characterize them after the gating mechanism was finalized.
- (v) Our multi-cycle test runs *three* cycles; longer continual-learning timescales may further differentiate the architectures.
- (vi) We use a *single* random seed per condition; small effects carry run-to-run variance we did not characterize directly.

We can, however, bound the seed-variance question with a back-of-the-envelope power calculation. The headline recognition signal is a difference of two proportions: tagged-fact MC accuracy ($p_T = 0.28$, $n_T = 134$) versus untagged ($p_U = 0.12$, $n_U = 66$). Treating each fact as an independent Bernoulli trial, the standard error of the difference is $\sqrt{p_T(1-p_T)/n_T + p_U(1-p_U)/n_U} \approx \sqrt{0.0015 + 0.0016} \approx 0.056$. The observed effect of +0.16 is therefore approximately 2.9 standard errors above zero (one-sided z -test, $p \approx 0.002$). A second seed could plausibly land ± 0.06 from the observed mean, but the qualitative conclusion—a positive recognition signal of order 0.10–0.20—should be robust. The Pareto curve and multi-cycle BCP results, where the absolute differences are larger ($\Delta\text{BCP} > 1.0$ between SLEEP-A and naive LoRA at every cycle), have wider safety margins. We acknowledge that this is a back-of-the-envelope analysis, not multi-seed empirical confirmation, and treat it as a budget item for follow-up work. The strongest claim our evidence supports is: in this specific configuration, on this dataset, with these safety hyperparameters, we observe the patterns reported.

Ethical considerations: Continual-learning systems for language models, if they work, would reduce retraining costs substantially and enable models to learn from interaction. They would also raise privacy and safety concerns: persistent memory of user inputs, model memorization of sensitive information, and asymmetric power between systems that can update themselves and users who cannot. Our specific architecture stores user inputs (tagged spans) in the model’s KV bank and consolidates a subset into permanent weights; deployed naively, this would amount to silent on-device data retention. We argue that any deployed continual-learning system must include explicit user controls over what is encoded, what is consolidated, and what is permanent, alongside mechanisms for selective forgetting on user request. These concerns are real but tractable; they do not, in our view, block the research direction.

9 Conclusion and Future Scope

We implemented and empirically evaluated a biologically-faithful memory consolidation architecture for transformer language models. Our experiments show that direct-write KV memory injection produces a measurable recognition signal in pretrained transformers but does not transfer to free-form recall without further mechanism; that the proposed consolidation step has no efficient single-cycle operating point yet does deliver multi-cycle preservation advantages over naive fine-tuning; and that five specific assumptions in the original formalization required empirical correction before the architecture would function at scale. The key open problem is consolidation: bridging from attention-based recognition into weights that support autoregressive generation, without sacrificing base-model preservation.

Future scope: Our results give follow-up work specific targets rather than vague directions:

- (i) *Retrieval-aware consolidation*: train the model to use stored memory during generation, in the manner of Memorizing Transformers, but as a phase within a multi-cycle consolidation pipeline rather than as a one-time fine-tune.
- (ii) *External validation criteria*: move from surprise-reduction (a proxy of learning) to direct recall accuracy (the target) inside the consolidation loop.
- (iii) *Adaptive safety mechanisms*: engage safety constraints only when degradation is detected, in the manner of trust-region methods, rather than applying them uniformly.
- (iv) *Multi-cycle evaluation at longer timescales*: run well beyond three cycles to test whether SLEEP’s BCP stabilizes or continues to compound.

We release the implementation, formalization, and full experimental logs to enable that work.

References

Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaisyasingam Ajanthan, Puneet K Dokania, Philip HS Torr, and Marc’Aurelio Ranzato. On tiny episodic memories in continual learning. In *ICML Workshop on Multi-Task and Lifelong Reinforcement Learning*, 2019.

- Susanne Diekelmann and Jan Born. The memory function of sleep. *Nature Reviews Neuroscience*, 11(2):114–126, 2010.
- Uwe Frey and Richard GM Morris. Synaptic tagging and long-term potentiation. *Nature*, 385(6616):533–536, 1997.
- Edward J Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. In *European Conference on Computer Vision*, 2017.
- James L McClelland, Bruce L McNaughton, and Randall C O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.
- Richard GM Morris. Elements of a neurobiological theory of hippocampal function: the role of synaptic tagging and capture. *European Journal of Neuroscience*, 23(11):2829–2846, 2006.
- Karim Nader, Glenn E Schafe, and Joseph E LeDoux. Fear memories require protein synthesis in the amygdala for reconsolidation after retrieval. *Nature*, 406(6797):722–726, 2000.
- Qwen Team. Qwen2.5 technical report, 2024. <https://qwenlm.github.io/blog/qwen2.5/>.
- Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *CVPR*, 2017.
- Roger L Redondo and Richard GM Morris. Making memories last: the synaptic tagging and capture hypothesis. *Nature Reviews Neuroscience*, 12(1):17–30, 2011.
- Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- Matthew A Wilson and Bruce L McNaughton. Reactivation of hippocampal ensemble memories during sleep. *Science*, 265(5172):676–679, 1994.
- Yuhuai Wu, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

Annexure A Formalization Amendments

We resolved 36 design questions in a pre-registered formalization document before any code was written. Empirical work surfaced five amendments, each documented with the experiment that motivated it. We summarize them here; the full formalization (including unchanged questions) is released alongside the implementation.

A.1 ($\delta_{\max}$ must scale with model size): The original specification of $\delta_{\max} = 0.001$ as a per-parameter clip bound saturates within ~ 30 training steps on a 7B model with rank-16 LoRA: 36/36 LoRA parameters hit the clip on every step thereafter, and validation always fails. Required correction: $\delta_{\max} \approx 0.01$ at 7B scale. The per-parameter bound is the wrong abstraction; what matters is the Frobenius norm of the LoRA update relative to the model’s natural weight scale. A scaling rule of $\delta_{\max}(d) = 0.001 \cdot \sqrt{d/768}$ partially corrects this but underestimates the empirically required value.

A.2 (α_{slow} must exceed the dtype precision floor): The original specification of $\alpha_{\text{slow}} = 10^{-5}$ produces gradient updates that round to zero in bfloat16 storage: training loss is completely flat across 100 steps. Bumping to 10^{-4} produces a phase transition (loss $1.46 \rightarrow 0.55$). Required correction: $\alpha_{\text{slow}} \geq 10 \cdot \text{ulp}(\text{dtype})$, which for bfloat16 is approximately $10^{-3} \cdot \|W\|_{\text{avg}}$. The original 10^{-5} implicitly assumed float32; in mixed-precision training (the practical regime for 7B+), it is too small.

A.3 (μ_{surprise} has a generator-discriminator gap): The quality check “replay must be at least as surprising as the model’s baseline” rejects all replays in practice, because replays are generated from the same model that defines the surprise function and are therefore systematically more in-distribution than external calibration text. The empirical surprise of generated replays (~ 1.25 nats) falls below the baseline computed on calibration text (~ 1.41 nats). Required correction: replace the absolute threshold with a relative-gain criterion ($\text{surprise}(\text{replay}) > \beta \cdot \text{surprise}(\text{seed})$) or compute the threshold from self-generated samples. Until amended, we disable the absolute check.

A.4 (Validation has a data-volume floor): The validation criterion $\text{surprise}_{\text{new}} < (1 - \epsilon_{\text{learn}}) \cdot \text{surprise}_{\text{old}}$ with $\epsilon_{\text{learn}} = 0.10$ is achievable only above a minimum number of replays. With 5 facts and 2–3 replays, 0% of candidates pass; with 200 facts and ~ 50 replays, 40%–60% pass depending on safety settings. The criterion itself is sound, but the formalization should explicitly document that pass rates are zero below a benchmark-dependent volume threshold.

A.5 (Tags are pointers, not storage): The original formalization stored only the K/V projections of tagged tokens, but tagged tokens are short novel-entity mentions (4–8 tokens) that capture the location of the surprise without its content. Required correction: when a tag fires, store K/V for the full sentence containing the tagged span (the *episode*), not just the tagged tokens. This corresponds to the proposal’s own framing of tags as pointers but was implemented as direct storage. Section 7.3 reports the empirical correction.

Annexure B Hyperparameter Table

Annexure C Implementation Details

C.1 KV Memory Injection Mechanics

KV memory injection requires care to avoid silently wrong attention scores. We document the choices made:

Position encoding. We treat memory entries as occupying negative position IDs $[-n_{\text{mem}}, -1]$. RoPE [Su et al., 2024] is applied at read time to memory keys with these positions. The encoding is mathematically valid (sin is odd, cos is even, both well-defined at negative inputs) and verified unit-tested. Memory thus appears to current queries (at positive positions) as “recent context tokens,” a pattern the model was trained on through RoPE’s relative-position mechanic.

Attention mask. Qwen2 uses float additive masks with $-\infty$ for masked positions and 0.0 for visible. When extending the mask with memory columns, we fill those columns with 0.0 of the mask’s own dtype. A boolean True or integer 1 here would silently produce wrong scores on float-mask attention paths.

Cache ordering. Memory K/V must be injected *after* the `past_key_values` cache update, not before. If memory is injected first, the cache accumulates a duplicate copy of memory tokens on every autoregressive generation step; by step 50 the cache contains 50 copies of memory and BCP collapses (we observed $\sim 96\times$ degradation under this bug).

Table 4: Full hyperparameter list. “Source” indicates which of the 36 formalization questions specifies the parameter. “Status” indicates whether the value matches the original specification or was empirically amended.

Parameter	Value	Source	Status
Tagging κ (z-score threshold)	1.5	Q1.2	default
Tagging g (gap tolerance)	3	Q1.3	default
Tagging min span length	4	Q1.3	default
Tagging EMA β	0.99	Q1.2	default
PRP weights (w_E, w_a, w_x, w_r)	(0.35, 0.30, 0.15, 0.20)	Q2.2	default
PRP budget multiplier c_{prp}	500	Q2.4	default
PRP cross-ref similarity threshold	0.5	Q2.2	default
LoRA rank (r)	16	Q3.1	default
LoRA α	32	Q3.1	default
Adapted layer fraction	0.333 (top 1/3)	Q3.2	default
Adapted matrices	V, O	Q3.1	default
α_{fast}	10^{-4}	Q3.3	default (LoRA W_{fast} only)
α_{slow}	10^{-4}	Q3.3	amended (A.2)
δ_{max}	0.01	Q3.4	amended (A.1)
λ_{EWC}	100	Q3.4	default
ϕ_{min} (plasticity floor)	0.10	Q3.4	default
ϵ_{learn} (validation)	0.10	Q4.6	default
$\epsilon_{\text{degrade}}$ (rollback)	0.02	Q4.5	default
Replay temperature	0.7	Q4.1	default
Replay top- p	0.9	Q4.1	default
Compression target	5	Q4.1	default
Steps per memory	5	Q4.4	default
Sleep weight decay	0.01	Q4.4	default
KV bank capacity (tokens)	20,000	—	new (A.5)
KV top- k retrieval gate	16 (default), 64 (episode)	—	new (A.5)
KV memory storage unit	full sentence (episode)	—	amended (A.5)
μ_{surprise} (quality check)	disabled (set to 0)	Q4.1	amended (A.3)

K/V extraction layer-norm. When extracting K/V from a forward pass via `output_hidden_states=True`, the apparent “hidden state at layer ℓ ” is the input to layer ℓ , but attention computes K/V from the layer-normed input. Skipping the layer-norm produces silently wrong K/V. We add an integration test that captures K/V via a forward hook during a normal forward pass and asserts equality with our extraction.

C.2 Per-Phase KV State

A correctness requirement we discovered: KV injection must be *disabled* during W_{cons} training. If left enabled, the model can satisfy the training loss by relying on injected memory rather than encoding the replay content into weights—a crutch the model would not have at inference time after the bank is cleared. We add a `set_enabled(bool)` method to the injector and toggle as follows:

- Wake (tagging, KV write): enabled.
- Phase 1 Generate (replay): enabled (so the model attends to source while compressing).
- Phase 2 Quality check: state preserved (no parameter changes; result invariant).
- Phase 3 Train (W_{cons} updates): *disabled* (forces W_{cons} to encode into weights).
- Phase 4 Perplexity check: disabled (clean preservation measurement).
- Phase 5 Validate & cleanup: disabled (clean recall measurement).

- After successful consolidation: bank is cleared, enabled flag reset for next cycle.

Annexure D Multi-Cycle Configuration

The multi-cycle experiment of Section 7.6 uses Setting A from the Pareto sweep ($\delta_{\max} = 0.02$, $\lambda_{\text{EWC}} = 50$, $\alpha_{\text{slow}} = 10^{-4}$, steps-per-memory = 4) for the SLEEP arm. The naive LoRA arm uses 200 gradient steps per cycle on the cycle’s batch only, with AdamW (lr 10^{-4} , weight decay 0.01, batch size 32, uniform-with-replacement sampling). Both arms accumulate adapter state across cycles; weights from cycle i persist into cycle $i + 1$. Both arms are evaluated on identical control texts and recall benchmarks after each cycle. Replay strategy for the SLEEP arm is set to `original` (replays are the original fact tokens), eliminating replay quality as a confound and isolating the multi-cycle dynamics of the consolidation training step.